

Reading One Line: the Catalog

Every option at every one of the seven questions, itemized, with the example form and what it means for the note

October 2026

How to use.

This is the lookup behind the seven-question method. At each question, find the row that matches the line in front of you; the last column tells you what to carry into the note or the next question. Rows are exhaustive for the code you will meet in a review; anything not listed is rare enough to look up.

Q1. What kind of line is it?

Declarations (a new label or type comes into existence)

#	Form	Example	Carry forward
1	<code>const</code> with value	<code>const a = 1;</code>	label fixed; go to Q2 on the right side
2	<code>let</code> with value	<code>let a = 1;</code>	label may be reassigned later
3	<code>let</code> without value	<code>let a; / let a: number;</code>	holds <code>undefined</code> until assigned
4	<code>var</code>	<code>var a = 1;</code>	function-scoped, hoisted; smell
5	destructuring object	<code>const { a, b } = o;</code>	two labels; each is a property read of <code>o</code>
6	destructuring with rename	<code>const { a: x } = o;</code>	label <code>x</code> holds <code>o.a</code>
7	destructuring with default	<code>const { a = 5 } = o;</code>	<code>a</code> is <code>o.a</code> unless <code>undefined</code> , then 5
8	destructuring rest	<code>const { a, ...rest } = o;</code>	<code>rest</code> is a NEW shallow object of the other keys
9	destructuring array	<code>const [x, y] = arr;</code>	<code>x</code> is <code>arr[0]</code> , <code>y</code> is <code>arr[1]</code> , either may be <code>undefined</code>
10	destructuring array rest	<code>const [h, ...t] = arr;</code>	<code>t</code> is a NEW array
11	function declaration	<code>function f(x: number) {}</code>	hoisted; a function object
12	async function declaration	<code>async function f() {}</code>	returns a Promise, always
13	generator	<code>function* g() {}</code>	returns an iterator
14	class	<code>class A {}</code>	a constructor; <code>new A()</code> makes instances
15	class field	<code>count = 0;</code> inside a class	per-instance property
16	class method	<code>m() {}</code> inside a class	on the prototype; <code>this</code> is the instance when called as <code>a.m()</code>
17	getter / setter	<code>get x() {} / set x(v) {}</code>	read as <code>a.x</code> ; no parentheses
18	static member	<code>static n = 0; / static f() {}</code>	on the class, not instances
19	<code>interface</code>	<code>interface U { id: number }</code>	type-only; runs nothing
20	<code>type</code> alias	<code>type Id = string number;</code>	type-only
21	<code>enum</code>	<code>enum C { A, B }</code>	a real object at runtime; numeric by default
22	<code>import</code>	<code>import { x } from ". / m";</code>	a label bound to another module's export
23	<code>export</code>	<code>export const a = 1; / export default f</code>	same as the declaration, plus visibility
24	<code>declare</code>	<code>declare const g: string;</code>	type-only; exists elsewhere

Assignments (an existing label re-pointed, or an object mutated)

#	Form	Example	Carry forward
25	reassignment	<code>a = 2;</code>	label re-pointed; needs <code>let</code>
26	compound	<code>a += 1; , a -= 1; , a *= 2; , s += "x";</code>	reassignment with arithmetic or concat
27	increment	<code>i++; , ++i; , i--;</code>	reassignment
28	logical assignment	<code>a = b; , a &&= b; , a ??= b;</code>	reassign only if falsy / truthy / nullish
29	property set	<code>o.k = v;</code>	MUTATES <code>o</code> ; every alias sees it
30	bracket set	<code>o["k"] = v; , arr[i] = v;</code>	mutates
31	nested set	<code>o.a.b = v;</code>	mutates <code>o.a</code> , which may be shared with something else

#	Form	Example	Carry forward
32	delete	<code>delete o.k;</code>	mutates
33	length set	<code>arr.length = 0;</code>	mutates; empties every alias
34	destructuring assignment	<code>({ a } = o); [x, y] = [y, x];</code>	reassigns existing labels
35	chained assignment	<code>a = b = 0;</code>	both labels point at 0

Expression statements (a bare expression; something runs, result dropped unless it mutates)

#	Form	Example	Carry forward
36	mutating method call	<code>arr.push(1); arr.sort(); map.set(k, v);</code>	the mutation is the effect; fine
37	non-mutating method call	<code>arr.map(f); s.trim(); arr.filter(p);</code>	result dropped; dead line
38	sync side-effect call	<code>console.log(x); res.json(d); emitter.emit(e);</code>	the side effect is the effect
39	async call, no await	<code>save(x); fetch(url);</code>	fire-and-forget; rejection unhandled
40	<code>await</code> alone	<code>await save(x);</code>	waits; value dropped on purpose
41	<code>new</code> alone	<code>new Worker(...);</code>	side effect of construction; object dropped
42	bare value	<code>x; a + b; o.k;</code>	does nothing
43	<code>void</code> expression	<code>void f();</code>	runs <code>f</code> , discards result; explicit fire-and-forget
44	IIFE	<code>(async () => { ... })();</code>	runs immediately; a Promise is dropped unless caught

Control (changes which line runs next)

#	Form	Example	Carry forward
45	<code>if</code> / <code>else if</code> / <code>else</code>	<code>if (x) {} else {}</code>	condition is coerced to boolean; check <code>=</code> vs <code>===</code>
46	ternary	<code>c ? a : b</code>	an expression, not a statement; has a value
47	<code>switch</code>	<code>switch (x) { case 1: ... }</code>	uses <code>===</code> ; missing <code>break</code> falls through; missing <code>default</code> returns <code>undefined</code>
48	<code>for</code>	<code>for (let i = 0; i < n; i++)</code>	check the bound; <code>let</code> per iteration
49	<code>for...of</code>	<code>for (const x of arr)</code>	values; supports <code>break</code> , <code>await</code>
50	<code>for...in</code>	<code>for (const k in obj)</code>	keys as strings; on arrays, indices as strings
51	<code>while</code> / <code>do...while</code>	<code>while (cond) {}</code>	check termination
52	<code>break</code> / <code>continue</code>		only in loops / <code>switch</code> ; not in <code>forEach</code>
53	<code>return</code>	<code>return x; / return;</code>	ends the function; <code>return;</code> is <code>undefined</code>
54	<code>throw</code>	<code>throw new Error("m");</code>	unwinds to the nearest <code>try</code> on the same stack
55	<code>try</code> / <code>catch</code> / <code>finally</code>	<code>try {} catch (e) {} finally {}</code>	catches only what is thrown or awaited inside
56	labeled statement	<code>outer: for (...)</code>	rare; <code>break outer</code>
57	<code>yield</code>	<code>yield x;</code>	inside a generator

Type-only (runs nothing; note what the author claims)

#	Form	Example	Claim
58	annotation	<code>x: number</code>	<code>x</code> is a number here
59	return type	<code>): Promise<User></code>	every path returns that
60	cast	<code>x as User</code>	trust me; no check
61	non-null	<code>x!</code>	not null; no check
62	<code>satisfies</code>	<code>o satisfies Config</code>	shape check without widening
63	generic parameter	<code><T></code>	placeholder
64	<code>as const</code>	<code>[...] as const</code>	literal types
65	<code>typeof x</code> in a type	<code>let y: typeof x</code>	same type as <code>x</code>
66	<code>keyof T</code>		union of key names
67	<code>@ts-ignore</code> / <code>@ts-expect-error</code>	comment	checking off for the next line; a finding

Q2. Where does the value come from?

Literals

#	Form	Example	Q3 answer
1	string	"a", 'a', `a \${x}`	primitive
2	number	42, 3.5, -0, 1e9, 0x1f, NaN, Infinity	primitive
3	boolean	true, false	primitive
4	nullish	null, undefined	primitive
5	bigint	10n	primitive
6	object literal	{}, { a: 1 }, { a }, { [k]: v }, { ...o }	object, NEW
7	array literal	[], [1, 2], [...a, ...b]	object, NEW
8	function literal	() => {}, function () {}, async () => {}	object (function), NEW
9	regex literal	/ab+/g	object, NEW
10	template literal	`Hello \${name}`	primitive string
11	new expression	new Map(), new Set(), new Date(), new Error(), new C()	object, NEW

Calls

#	Form	Example	Q4 applies?
12	plain function call	f(x)	yes: is f async?
13	method call	obj.m(x), arr.map(f), s.trim()	yes if the method is async; also note mutate vs new
14	optional call	cb?.(x)	may not run at all
15	constructor	new C(x)	not a Promise (unless C is Promise)
16	chained calls	a.b().c().d()	each . on the previous result; the last one's return is the value
17	IIFE	((() => 5)())	the arrow's return
18	tagged template	sql`...`	a function call
19	await of a call	await f(x)	Q4 answered: unwrapped
20	.then chain	f().then(g)	a Promise of g's result
21	Promise.all([...])		a Promise of an array
22	JSON.parse(s)		sync; any; throws on bad input
23	Number(x), String(x), Boolean(x)		conversion; Number may give NaN
24	parseInt(s, 10), parseFloat(s)		partial parse; NaN on failure
25	Object.keys/values/entries(o)		new arrays
26	Array.from(x), Array.isArray(x)		new array / boolean
27	structuredClone(o)		deep copy, new
28	Date.now()		primitive number
29	Math.*	Math.max(...arr)	number; -Infinity on empty spread

Property and element reads

#	Form	Example	Can be nothing?
30	dot	o.k	yes if k is optional or absent
31	bracket, string key	o["k"], o[name]	yes; also prototype keys on plain objects
32	index	arr[i], arr[0], arr[arr.length - 1]	yes past the end; arr[-1] is always undefined
33	arr.at(i)		yes; -1 works
34	arr.length, s.length		number, never undefined
35	map.size, set.size		number
36	map.get(k)		yes when absent

#	Form	Example	Can be nothing?
37	optional chain	<code>o?.k</code> , <code>o?.[i]</code> , <code>o?.k?.j</code>	yes; by design
38	nested	<code>o.a.b.c</code>	crashes if any link is nothing
39	request fields	<code>req.params.id</code> , <code>req.query.x</code> , <code>req.body.x</code> , <code>req.headers["x"]</code>	yes; and the type is string / string[] / any
40	<code>this.field</code>		depends on how the method was called
41	<code>process.env.X</code>		<code>string</code> <code>undefined</code>
42	<code>localStorage.getItem(k)</code>		<code>string</code> <code>null</code>

Operators

#	Form	Example	Result
43	arithmetic	<code>a + b</code> , <code>a - b</code> , <code>a * b</code> , <code>a / b</code> , <code>a % b</code> , <code>a ** b</code>	number; <code>+</code> concatenates if either side is a string; <code>/ 0</code> is <code>Infinity</code> ; bad input is <code>NaN</code>
44	string concat	<code>"a" + x</code>	string; <code>x</code> coerced
45	comparison	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	boolean; <code>NaN</code> makes all <code>false</code> ; check the boundary
46	equality	<code>===</code> , <code>!==</code>	boolean, strict
47	loose equality	<code>==</code> , <code>!=</code>	boolean, coerced; flag
48	logical AND	<code>a && b</code>	<code>a</code> if falsy, else <code>b</code>
49	logical OR	<code>a b</code>	<code>a</code> if truthy, else <code>b</code> ; replaces <code>0</code> , <code>""</code> , <code>false</code>
50	nullish	<code>a ?? b</code>	<code>a</code> unless <code>null</code> / <code>undefined</code>
51	NOT	<code>!x</code> , <code>!!x</code>	boolean; <code>!x</code> is true for <code>0</code> , <code>""</code> , <code>[]</code> is NOT falsy
52	ternary	<code>c ? a : b</code>	<code>a</code> or <code>b</code>
53	spread in literal	<code>[...a]</code> , <code>{ ...o }</code>	new container, shallow
54	<code>typeof x</code>		a string naming the type
55	<code>x instanceof C</code>		boolean
56	<code>k in o</code>		boolean; includes prototype keys
57	<code>delete o.k</code>		boolean; mutates
58	<code>void x</code>		<code>undefined</code>
59	comma	<code>(a, b)</code>	<code>b</code> ; rare
60	assignment as expression	<code>if (x = 5)</code>	the assigned value; always a bug in a condition
61	bitwise	<code>a & b</code> , <code>a b</code> , <code>a ^ b</code> , <code>~a</code> , <code>a << 1</code>	integer math; rare; <code> </code> in a value position is bitwise, in a type position is union

await and destructuring

#	Form	Example	Result
62	<code>await p</code>		the resolved value; throws on rejection
63	<code>await nonPromise</code>	<code>await 5</code>	<code>5</code> ; harmless; still yields
64	<code>const { a } = o</code>		<code>o.a</code> ; may be <code>undefined</code>
65	<code>const [x] = arr</code>		<code>arr[0]</code>
66	<code>const { a: { b } } = o</code>		<code>o.a.b</code> ; crashes if <code>o.a</code> is nothing
67	<code>for (const [k, v] of map)</code>		each entry
68	<code>for (const [k, v] of Object.entries(o))</code>		each key-value pair

Q3. Primitive or object?

Primitives (immutable, by value, copied, no keys)

#	Type	Produced by
1	<code>string</code>	string literal, template, every string method, <code>String(x)</code> , <code>JSON.stringify</code> , <code>s.join</code> , <code>typeof</code>
2	<code>number</code>	numeric literal, arithmetic, <code>Number(x)</code> , <code>parseInt</code> , <code>arr.length</code> , <code>map.size</code> , <code>Date.now()</code> , <code>arr.indexOf</code> , <code>arr.push</code> (returns length), <code>Math.*</code>

#	Type	Produced by
3	boolean	true/false, comparisons, ===, !x, arr.includes, arr.some/every, map.has, Array.isArray, instanceof, in, delete, set.delete
4	undefined	let x;, missing key, arr[i] past end, map.get miss, find miss, no return, void, o?.k short-circuit
5	null	null literal, DB not-found, JSON.parse("null"), localStorage.getItem miss, regex.exec miss
6	bigint	10n, BigInt(x)
7	symbol	Symbol(), Symbol.iterator

Objects (mutable, by identity, aliased, have keys)

#	Kind	Produced by
8	plain object	{ } literal, { ...o }, Object.assign, JSON.parse, Object.fromEntries, req.body, req.query
9	array	[] literal, [...a], map, filter, slice, concat, flat, split, Object.keys/values/entries, Array.from, toSorted
10	Map / Set	new Map(), new Set(), new Map(m), Map.groupBy
11	function	function literal, arrow, method reference obj.m, fn.bind(x)
12	Promise	any async call, fetch, res.json, new Promise, .then, Promise.all
13	Date	new Date()
14	Error	new Error(), catch (e) (as unknown)
15	RegExp	/x/, new RegExp
16	class instance	new C()
17	iterator	arr.entries(), map.keys(), generator call
18	Response / Request / Headers / URL	fetch, new URL(s)
19	Buffer / typed array	Buffer.from, new Uint8Array
20	req / res / client / emitter	framework-provided

Q4. Ticket or value?

Returns a Promise (a ticket)

#	Form	Note
1	any function declared async	guaranteed
2	any function annotated : Promise<...>	guaranteed
3	fetch(url)	resolves on any HTTP status
4	res.json(), res.text(), res.blob(), res.arrayBuffer()	
5	new Promise(...), Promise.resolve, Promise.reject	
6	Promise.all, allSettled, race, any	
7	p.then(f), p.catch(f), p.finally(f)	chaining always yields a Promise
8	import("./m")	dynamic import
9	fs/promises anything, timers/promises, stream/promises	
10	util.promisify(fn)(...)	
11	db.query, pool.query, prisma.*, Model.find, collection.insertOne, redis.get, knex(...) awaited	every DB client
12	axios.get, got, ky, client.send(cmd)	every HTTP client / SDK
13	a plain function whose body returns any of the above	pass-through
14	arr.map(async x => ...)	an ARRAY of Promises; needs Promise.all
15	an async IIFE	

Returns a value (not a ticket)

#	Form	Note
16	<code>JSON.parse</code> , <code>JSON.stringify</code>	sync
17	all array methods (<code>map</code> , <code>filter</code> , <code>find</code> , <code>sort</code> , <code>push</code> ...)	sync; the callbacks may be async but the method is not
18	all string methods	sync
19	<code>Object.*</code> , <code>Math.*</code> , <code>Number(...)</code> , <code>Date.now()</code>	sync
20	<code>fs.readFileSync</code> and every <code>*Sync</code>	sync, blocking
21	<code>setTimeout</code> , <code>setInterval</code>	return an id, not a Promise
22	<code>res.json(d)</code> , <code>res.send</code> , <code>res.status</code> (Express)	sync; chainable; do not end the function
23	<code>localStorage.*</code> , <code>console.*</code> , <code>crypto.randomUUID</code>	sync
24	<code>new C()</code> for any ordinary class	sync
25	<code>arr.forEach(async ...)</code>	returns <code>undefined</code> ; the Promises are dropped

How the ticket is handled on this line

#	Form	Verdict
26	<code>const v = await f()</code>	unwrapped; <code>v</code> is the value
27	<code>return f()</code> inside <code>async</code>	handed to the caller; not caught by this <code>try</code>
28	<code>return await f()</code>	unwrapped then returned; caught by this <code>try</code>
29	<code>f().then(g).catch(h)</code>	handled via chain; the chain's result still needs catching
30	<code>await Promise.all(arr)</code>	all unwrapped into an array
31	<code>const p = f()</code> then <code>await p</code> later	deferred; fine if the <code>await</code> comes before the value is used and inside the right <code>try</code>
32	<code>f();</code>	dropped; fire-and-forget
33	<code>const v = f()</code> then <code>v.x</code>	ticket used as value; <code>undefined</code>
34	<code>await</code> inside <code>forEach</code> callback	not waited for by anyone
35	<code>await</code> inside a non-async function	syntax error

Q5. Can it be nothing?

Yes, it can be `undefined`

#	Source	Example
1	optional property	<code>o.a</code> where <code>a?: T</code>
2	absent property on a loose type	<code>(o as any).x</code> , <code>JSON.parse(s).x</code>
3	<code>arr.find</code> , <code>findLast</code>	no match
4	<code>arr[i]</code> , <code>arr.at(i)</code>	out of range
5	<code>arr[arr.length]</code> , <code>arr[-1]</code>	always
6	<code>map.get(k)</code> , <code>weakMap.get(k)</code>	absent key
7	<code>arr.pop()</code> , <code>arr.shift()</code>	empty array
8	<code>req.headers[k]</code> , <code>req.query.k</code> , <code>req.params.k</code> , <code>req.body.k</code>	not sent
9	<code>process.env.X</code>	not set
10	a function with a fall-off path	no <code>return</code> on that path
11	<code>=> { ... }</code> with no <code>return</code>	always
12	<code>let x;</code> before assignment	
13	any <code>o?.k</code>	short-circuit
14	destructuring a missing key	<code>const { a } = {}</code>
15	default param when caller passes <code>undefined</code> explicitly	default applies, so NOT nothing; but passes <code>null</code> → <code>null</code>
16	<code>Object.values(o)[0]</code>	empty object
17	<code>str.match(re)?.[1]</code>	no match / no group
18	<code>Promise.race([])</code>	never settles; rare

Yes, it can be `null`

#	Source	Example
19	explicit <code>null</code>	<code>let x = null</code>
20	DB row lookup	<code>findOne</code> → <code>null</code>
21	<code>JSON.parse("null")</code>	
22	<code>localStorage.getItem(k)</code>	miss
23	<code>regex.exec(s)</code> , <code>s.match(re)</code>	no match
24	<code>document.querySelector</code>	browser; not found
25	a <code>T null</code> return type	by declaration
26	a default param when the caller passes <code>null</code>	default does not apply

No, it cannot be nothing (unless the type lies)

#	Source
27	a literal
28	<code>arr.length</code> , <code>s.length</code> , <code>map.size</code> (a number)
29	<code>map</code> , <code>filter</code> , <code>slice</code> , <code>Object.keys</code> (an array, possibly empty)
30	<code>new C()</code>
31	<code>Number(x)</code> (a number, possibly <code>NaN</code> , never <code>undefined</code>)
32	<code>String(x)</code> , template literals
33	<code>arr.includes</code> , <code>map.has</code> , comparisons (a boolean)
34	<code>await f()</code> where <code>f</code> is declared <code>Promise<T></code> with <code>T</code> non-nullable, and the declaration is honest

How the next use handles nothing

#	Form	Handles <code>undefined</code> ?	Handles <code>null</code> ?	Also catches <code>0</code> / <code>""</code> / <code>false</code> ?
35	<code>if (x) / x && x.k</code>	yes	yes	YES (wrong for counts, flags)
36	<code>if (!x) return</code>	yes	yes	YES
37	<code>if (x === undefined)</code>	yes	no	no
38	<code>if (x == null)</code>	yes	yes	no (the one idiomatic <code>==</code>)
39	<code>x ?? d</code>	yes	yes	no
40	<code>x d</code>	yes	yes	YES
41	<code>x?.k</code>	yes	yes	no
42	<code>x!.k</code>	claims no; checks nothing		
43	<code>x.k</code>	not handled; crashes		
44	<code>typeof x === "string"</code>	yes (it is not a string)	yes	no
45	<code>Array.isArray(x)</code>	yes	yes	no
46	<code>x instanceof C</code>	yes	yes	no
47	<code>"k" in x</code>	crashes on <code>undefined</code> / <code>null</code>		

Q6. Who else points here?

This line made a NEW address (own)

#	Form
1	object literal <code>{}</code> , <code>{ a: 1 }</code>
2	array literal <code>[]</code> , <code>[1, 2]</code>
3	<code>{ ...o }</code> , <code>{ ...a, ...b }</code> (new outer; inner objects shared)
4	<code>[...a]</code> , <code>[...a, ...b]</code> (new outer; elements shared)
5	<code>a.slice()</code> , <code>a.slice(i, j)</code> , <code>Array.from(a)</code> , <code>a.concat(b)</code>
6	<code>a.map(f)</code> , <code>a.filter(p)</code> , <code>a.flat()</code> , <code>a.flatMap(f)</code> (elements shared)
7	<code>a.toSorted()</code> , <code>a.toReversed()</code> , <code>a.toSpliced()</code> , <code>a.with(i, v)</code>

#	Form
8	<code>Object.assign({}, o), Object.fromEntries(...), Object.keys/values/entries(o)</code>
9	<code>new Map(m), new Set(s), new Map(), new Set()</code>
10	<code>structuredClone(o)</code> (deep)
11	<code>JSON.parse(JSON.stringify(o))</code> (deep-ish)
12	<code>new C(...), new Date(), new Error()</code>
13	a function literal
14	<code>s.split(...), Object.groupBy(...), Map.groupBy(...)</code>
15	the return of any function that builds and returns a literal

This line made an ALIAS (shared with something else)

#	Form	Shared with
16	<code>const b = a;</code>	<code>a</code>
17	a function parameter	the caller's argument
18	<code>arr.find(p), arr[i], arr.at(i), arr[0]</code>	the element inside <code>arr</code>
19	<code>map.get(k)</code>	the value inside the map
20	<code>o.k, o.a.b</code> where the value is an object	the nested object inside <code>o</code>
21	<code>const { inner } = o</code>	<code>o.inner</code>
22	<code>return this.items / return cache.get(k)</code>	internal state, now held by the caller
23	<code>cache.set(k, obj)</code> then handing <code>obj</code> out	the cache and every recipient
24	a closure referencing an outer object	the outer scope
25	<code>a.sort(), a.reverse(), a.fill()</code> (return the same array)	<code>a</code>
26	<code>Object.assign(target, src)</code> returns <code>target</code>	<code>target</code>
27	elements of <code>a.filter(p), a.map(x => x)</code>	the original elements
28	<code>arr.forEach((o) => ...)</code> callback parameter	the element
29	a module-level <code>const state = {}</code> referenced in a function	every caller of that module
30	<code>this</code> inside a method	the instance

Mutations that reach every alias

#	Form
31	<code>o.k = v, o[k] = v, arr[i] = v</code>
32	<code>delete o.k</code>
33	<code>arr.push, pop, shift, unshift, splice</code>
34	<code>arr.sort, reverse, fill, copyWithin</code>
35	<code>arr.length = n</code>
36	<code>map.set, map.delete, map.clear; set.add, set.delete, set.clear</code>
37	<code>date.setFullYear/Month/Date/Hours/Time</code>
38	<code>Object.assign(o, src)</code> (first argument)
39	<code>Object.freeze(o), Object.defineProperty(o, ...)</code>
40	<code>o.nested.k = v</code> (mutates the nested object; reaches everyone sharing <code>o.nested</code>)
41	<code>arr.forEach((o) => o.k = v)</code> (mutates each element)
42	<code>regex.exec(s)</code> with the <code>g</code> flag (advances <code>lastIndex</code>)

Who the “else” usually is

#	Holder	How to spot
43	the caller	the object came in as a parameter
44	the module / every request	a <code>const x = new Map()</code> or <code>{}</code> at module scope
45	the class / every method	<code>this.items</code>
46	the cache	<code>cache.get(k)</code> returned it

#	Holder	How to spot
47	the original array	the object was an element from <code>find</code> / <code>[i]</code> / <code>filter</code>
48	a previous caller	the function returned the same object before
49	a listener or timer	a closure captured it
50	<code>defaults</code> / config	<code>const opts = defaults</code> instead of <code>{ ...defaults }</code>

Q7. Was the result kept?

Kept (the value goes somewhere)

#	Form
1	assigned: <code>const x = ...</code> , <code>x = ...</code> , <code>o.k = ...</code>
2	returned: <code>return ...</code>
3	awaited into a label: <code>const x = await ...</code>
4	passed as an argument: <code>f(...)</code> , <code>arr.push(...)</code> , <code>res.json(...)</code>
5	used in a condition: <code>if (...)</code> , <code>while (...)</code> , <code>c ? ... : ...</code>
6	used in an expression: <code>a + f()</code> , <code>`\${f()}`</code>
7	stored: <code>map.set(k, ...)</code> , <code>arr[i] = ...</code>
8	spread into a literal: <code>[...f()]</code> , <code>{ ...f() }</code>
9	destructured: <code>const { a } = f()</code>
10	chained: <code>f().g()</code> (the next call consumes it)
11	yielded: <code>yield ...</code>
12	thrown: <code>throw ...</code>

Not kept, and that is fine (the effect was the point)

#	Form
13	mutating call: <code>arr.push(x);</code> , <code>arr.sort();</code> , <code>map.set(k, v);</code> , <code>set.add(x);</code> , <code>arr.splice(i, 1);</code>
14	property set: <code>o.k = v;</code>
15	side-effect call: <code>console.log(x);</code> , <code>res.json(d);</code> , <code>emitter.emit(e);</code> , <code>logger.info(...)</code> , <code>clearTimeout(id);</code>
16	<code>await save(x);</code> (waited; result intentionally dropped)
17	<code>void f();</code> (explicit discard)
18	<code>f(x);</code> where <code>f</code> is sync and documented to have side effects

Not kept, and that is the bug

#	Form	Why
19	<code>arr.map(f);</code> , <code>arr.filter(p);</code> , <code>arr.slice();</code> , <code>arr.concat(b);</code> , <code>arr.flat();</code>	new array dropped; dead line
20	<code>arr.toSorted();</code> , <code>arr.toReversed();</code>	new array dropped
21	<code>s.trim();</code> , <code>s.toUpperCase();</code> , <code>s.replace(...);</code> , <code>s.slice(...);</code>	new string dropped
22	<code>Object.assign({}, o);</code> , <code>{ ...o }</code> , <code>[...a];</code>	copy dropped
23	<code>structuredClone(o);</code> , <code>JSON.parse(s);</code>	result dropped
24	<code>arr.find(p);</code> , <code>arr.includes(x);</code> , <code>map.get(k);</code> , <code>map.has(k);</code>	lookup dropped
25	<code>f();</code> where <code>f</code> is async	Promise dropped; rejection unhandled
26	<code>fetch(url);</code>	same
27	<code>arr.map(async x => ...);</code>	array of Promises dropped
28	<code>p.then(g);</code> with no <code>.catch</code> and not awaited	chain dropped
29	<code>x;</code> , <code>a + b;</code> , <code>o.k;</code> , <code>arr[0];</code>	pure expression; nothing happens
30	<code>new Date();</code> , <code>new User();</code>	object dropped
31	<code>(async () => { ... })();</code> with no <code>.catch</code>	Promise dropped
32	<code>const sorted = arr.sort();</code>	kept, but NOT a copy: <code>sorted === arr</code> ; the keeping misleads

#	Form	Why
33	<code>const n = arr.push(x);</code>	kept the length, not the array

The note, with every field

After Q7, write:

label holds: · <own | shared with X> · · <ticket | value>

.

Field	Options
value	a concrete example: <code>6</code> , <code>"ABC"</code> , <code>{ id: "42", name: "Ann" }</code> , <code>[1, 10, 9]</code> , <code>Promise<User></code> , <code>undefined</code>
own / shared	own (new address); shared with: the caller / <code>users[]</code> / the cache / module state / <code>defaults</code>
can be nothing	yes (why: find miss, optional key, header absent) / no
ticket / value	value / ticket, not awaited / ticket, awaited on line N

The next line's Q5 reads “can be nothing”; Q6 reads “own / shared”; Q4 reads “ticket / value”. That is the entire hand-off.